

Software architecture

Software architecture

Rodrigo Martins Pagliares

Fundamental software architecture concepts

Introduction to software architecture

Lecture: Introduction to software architecture

Module: Fundamental software architecture concepts

What is software architecture?

What is software architecture?

“The manner in which the components of a computer or computer system are organized and integrated”.

(Merriam-Webster)

<https://www.merriam-webster.com/dictionary/architecture>

**The goal of
software architecture**

The goal of software architecture

“The goal of software architecture is to minimize the human resources required to build and maintain the required system”.

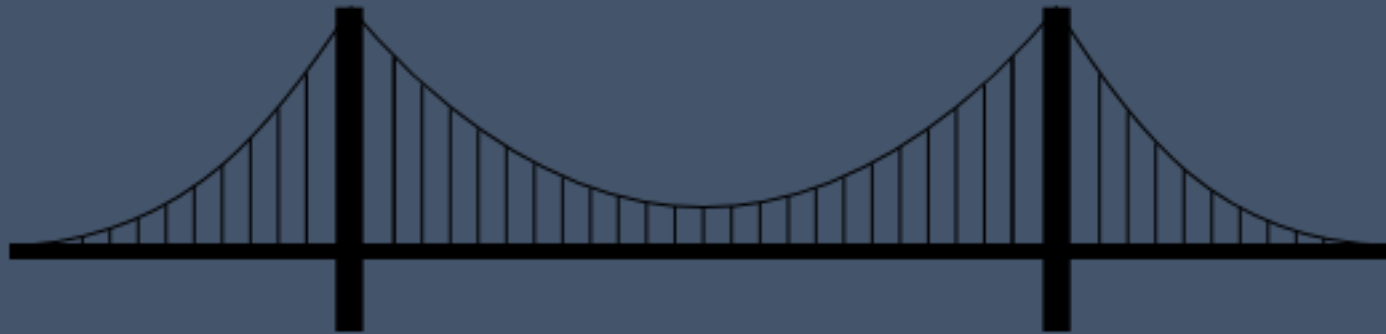
(Martin, R. Clean architecture, page 5)



The importance of software architecture

(for developers and business owners)

Business experts typically aren't technologists.
They **know how** to run their **business**, but they **don't necessarily know** much about **IT systems**.



Importance of architecture

“The dilemma for software developers is that business managers are not equipped to evaluate the importance of architecture”.

(Martin, R. The clean architecture, page 17)



Why do we need a software architecture?

Sometimes a **developer** just wants to **jump right in** and **start coding**.

Why do we need a software architecture?

- IT projects are complex, expensive and risky.
 - ✓ Must be build over a solid foundation to not negatively impact the quality of what is build on top of it.



Source: <https://bit.ly/3rFG2vF>

Why do we need a software architecture?

- Facilitates communication among stakeholders.



Why do we need a software architecture?

- Increases development team productivity.



Source: <https://bit.ly/3aQ09Pb>

Architecture and the development process

Architecture and the development process

- Architects must begin their efforts early in a development process.
- If architecture comes last, then the system will become ever more costly to develop.
 - ✓ Change will become practically impossible for part or all of the system.



Architecture vs. design

Lecture: Architecture vs. design

Module: Fundamental software architecture concepts

Architecture vs. design

Architecture vs. design

*“There is no difference between **architecture** and **design**. None at all”.*
(Martin, R. Clean architecture, page 5)



Architecture vs. design

- **Design** provides a set of components to implement a functional requirement.
 - ✓ Use case, use story, etc..
- **Architecture** provides a template into which the designed components are realized.
 - ✓ Normally associated with non-functional requirements.

Software Architect vs. Software Designer

Software architects

- **Software architects** are more **focused** on the **structure** of the **system** than of its **features** and **functions**.
- **Architects** create an **architecture** that allows those **features** and **functions** to be **easily developed**, **modified**, and **extended**.

Software Architect vs. Software Designer

	Software Architect	Software Designer
Abstraction level	- High/broad - Focus on few details	- Low/specific - Focus on many details
Deliverables	- System and subsystem - Architectural prototypes	- Component designs - Code specification
Area of focus	- Non Functional Requirements - Risk management	- Functional Requirements

The Software Architect role and responsibilities

Lecture: The Software Architect role and responsibilities

Module: Fundamental software architecture concepts

Software Architect roles

A myriad of roles

...

Software architect

Enterprise architect

Embedded software architect

Application Architect

System architect

...

Solution Architect

Cloud Architect

...

Software Architect's responsibilities

**A software architect usually has
technical and management responsibilities.**

Architect's technology responsibilities

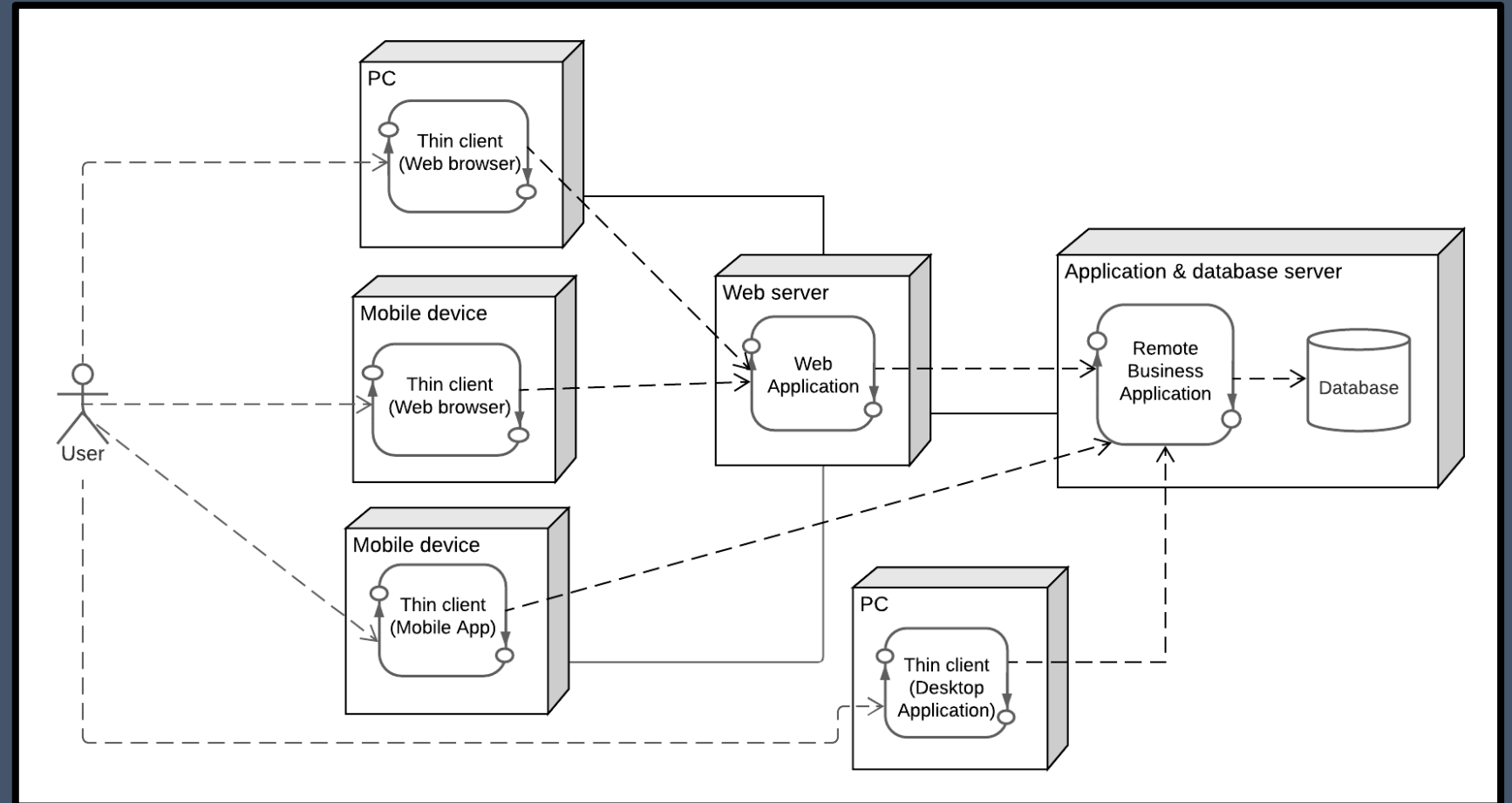
Architect's technology responsibilities

- Identify risks and create mitigation plans.
- List software assumptions and constraints.



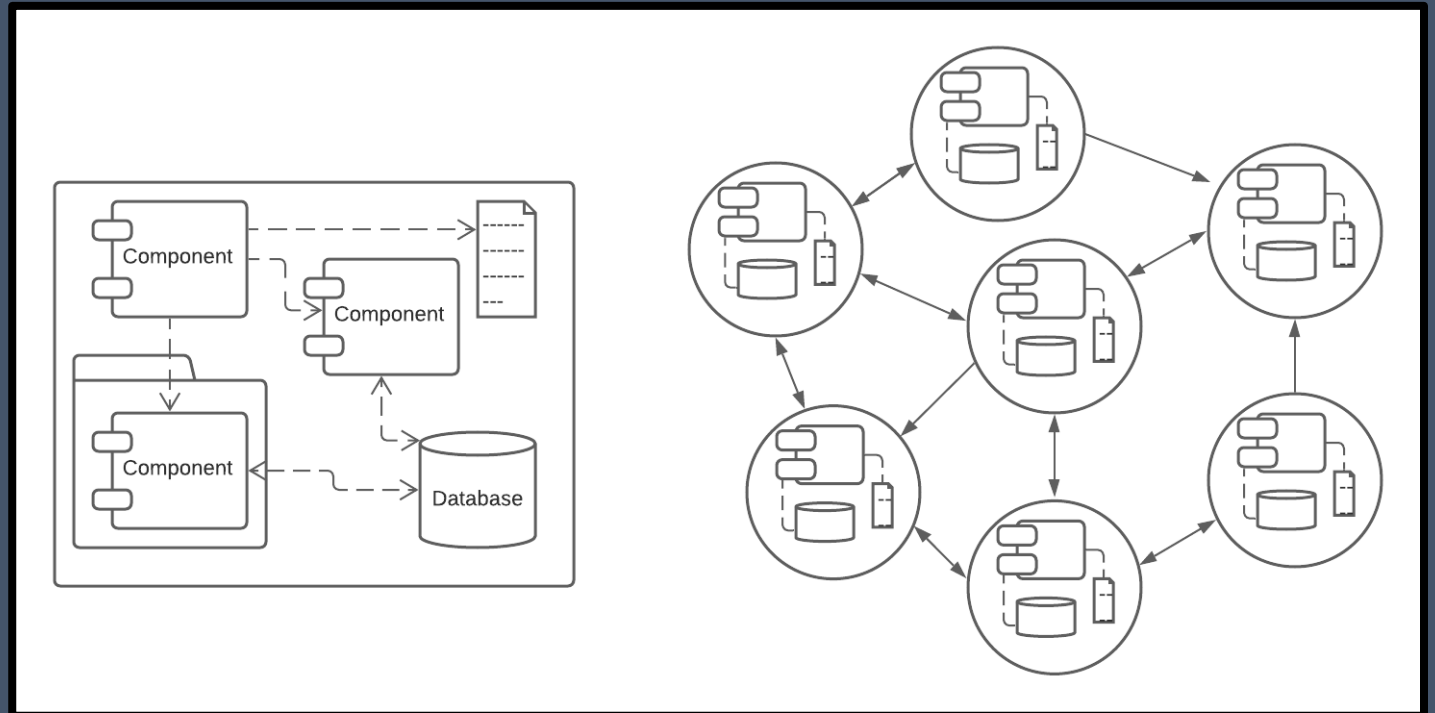
Architect's technology responsibilities

- Describe the deployment environment.



Architect's technology responsibilities

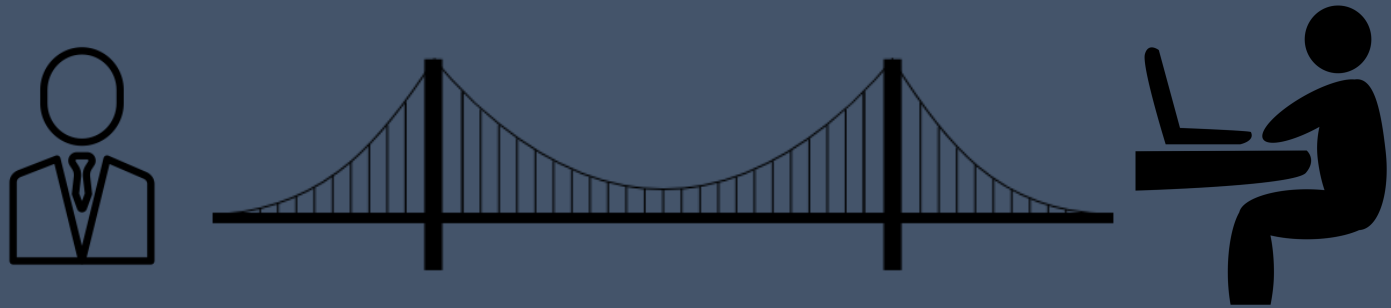
- Create documentation and interaction diagrams.
- Strive to define modules, components, and services that are easily testable (falsifiable).



Architect's management responsibilities

Architect's management responsibilities

- **Convince other stakeholders of the validity of decisions.**
- **Align strategic business processes and tech strategy.**
 - ✓ Framing solution proposals In terms that make sense to both business and technical stakeholders.
- **Manage relationships.**
 - ✓ Must have verbal and written communication.



Architect's management responsibilities

- **Mentor other team members.**



Architect's management responsibilities

- Understand **business strategy** and investigate **business requirements**.



A **day** in the **life** of an **architect** ranges
across many **different tasks**.

Why do we need the software architect role?

Lecture: Why do we need the software architect role?

Module: Fundamental software architecture concepts

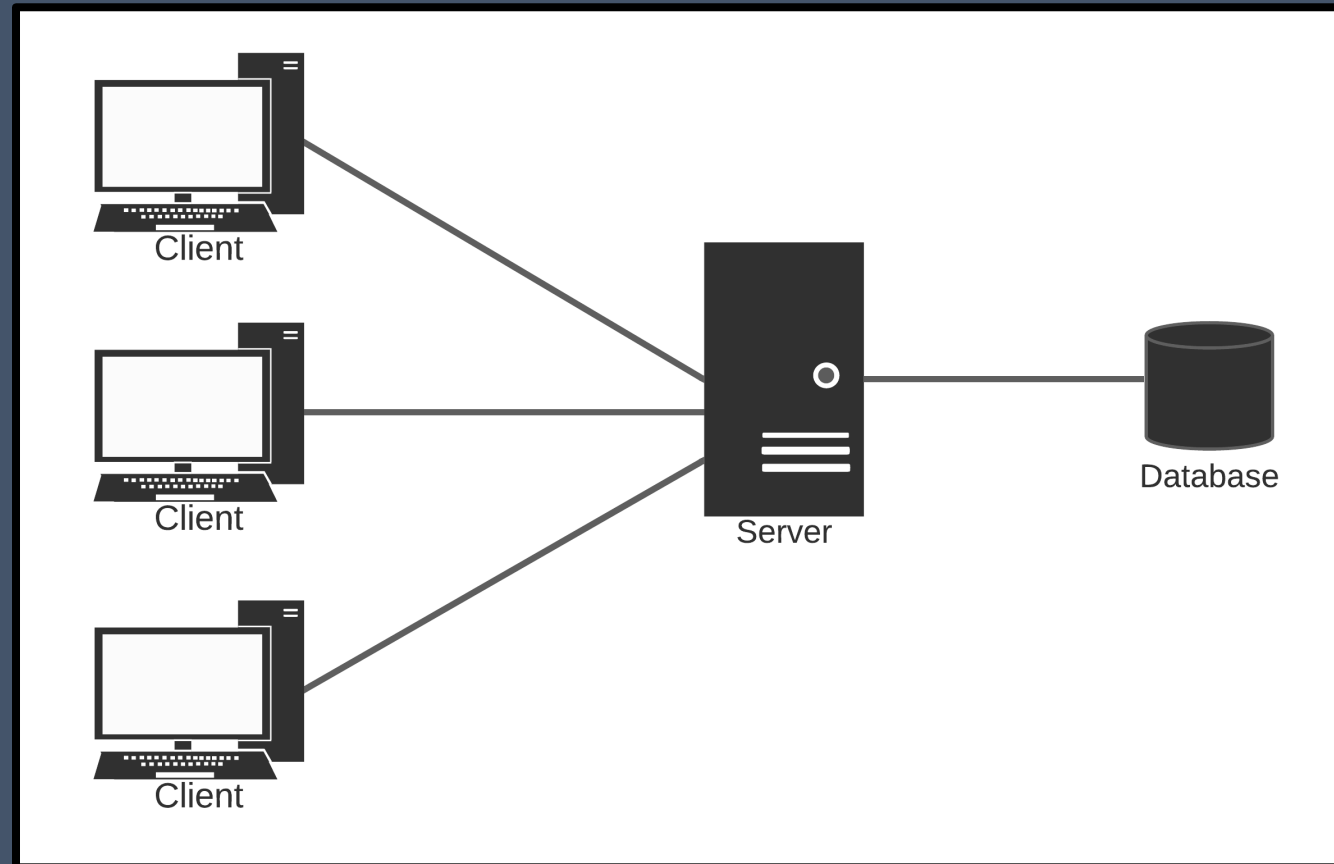
**Why do we need the
architect role?**

Why do we need the Architect role?

- **Software engineering** is employing **Software Architects** because of:
 - ✓ **Complexity, scale and distribution.**
- **Software systems we build today** ranges from **systems** that are **deployed on minimally** to **highly distributed systems** with a **wide range of scalability requirements.**
 - ✓ **On premises and on the cloud.**

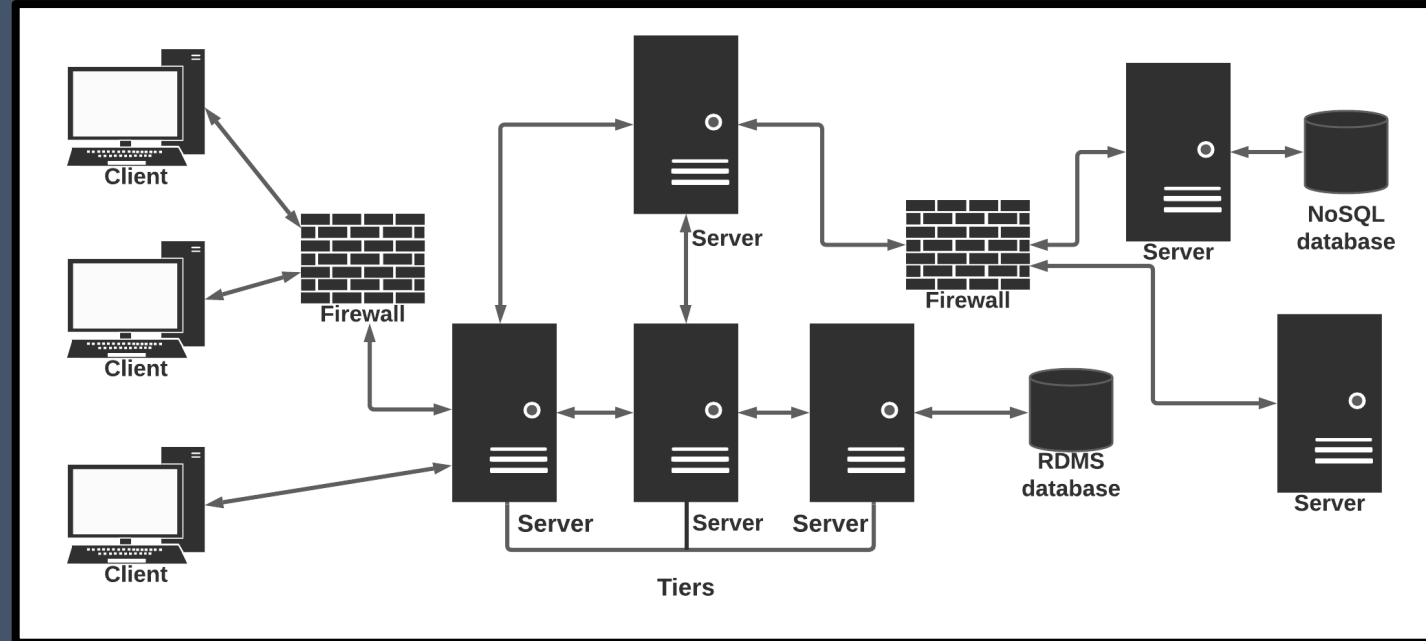
Minimally-distributed systems

- Typical client-server architecture.
- Lower risks when compared to large-scale, distributed enterprise systems.
- Often used for simple software solutions.
 - Low-demand for software architects.



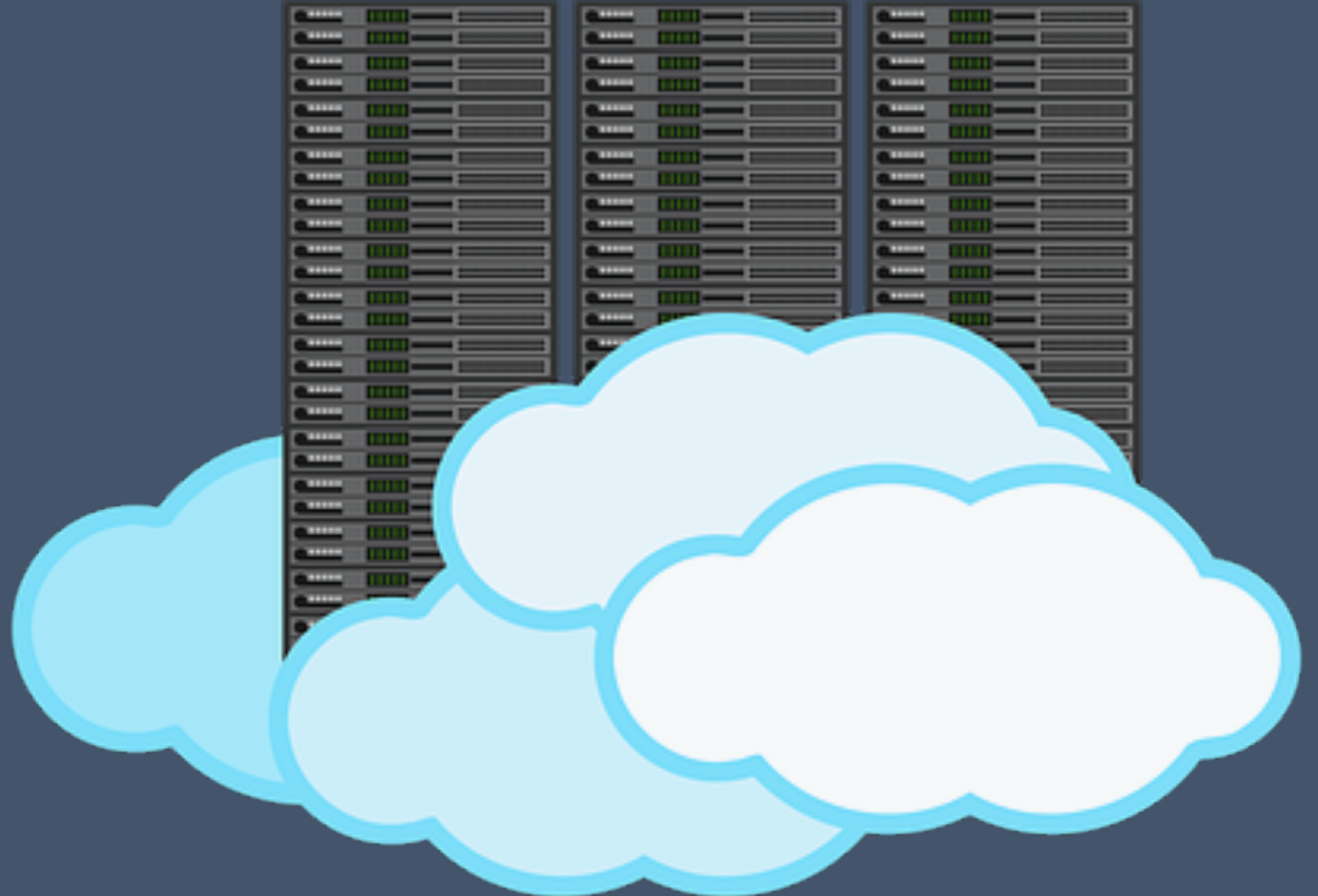
Highly-distributed systems

- Several risks are associated with large-scale, distributed enterprise systems.
- Typical scenario for complex software solutions.
 - High-demand for software architects.



Cloud computing

High-demand for
software (cloud)
architects.



Layers and Tiers

Lecture: Layers and tiers

Module: Fundamental software architecture concepts

Layers & tiers

- Layers

*“describe the **logical groupings** of the **functionality** and **components** in an **application**”.*

- Tiers

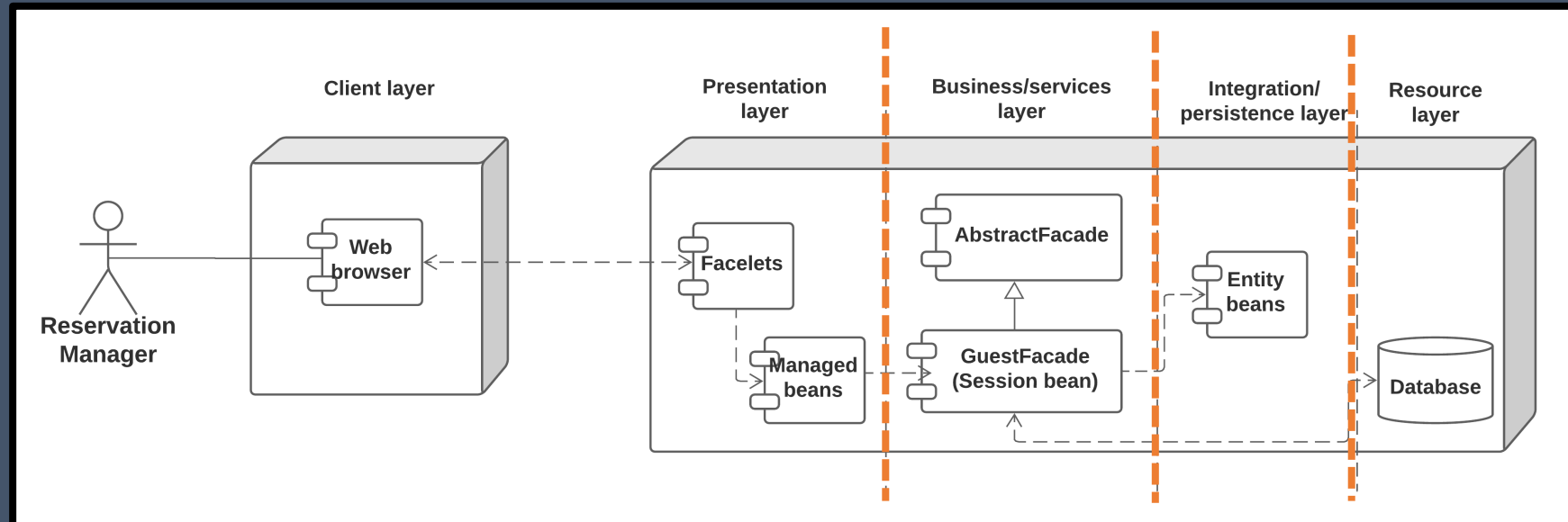
*“describe the **physical distribution** of the **functionality** and **components** on **separate servers, computers, networks, or remote locations**.”*

Layers & tiers

- **Logical layers** are merely a **way** of **organizing** your **source-code**.
 - ✓ For example, **Presentation layer**, **Business layer** and **Data layer**.
 - ✓ It is **not implied** that these **layers** might run on **different computers** or in **different processes** on a **single computer** or **even** in a **single process** on a **single computer**.
- **Physical tiers** are about **where** the **code runs**.
 - ✓ Places where **layers** are **deployed** and **run**.

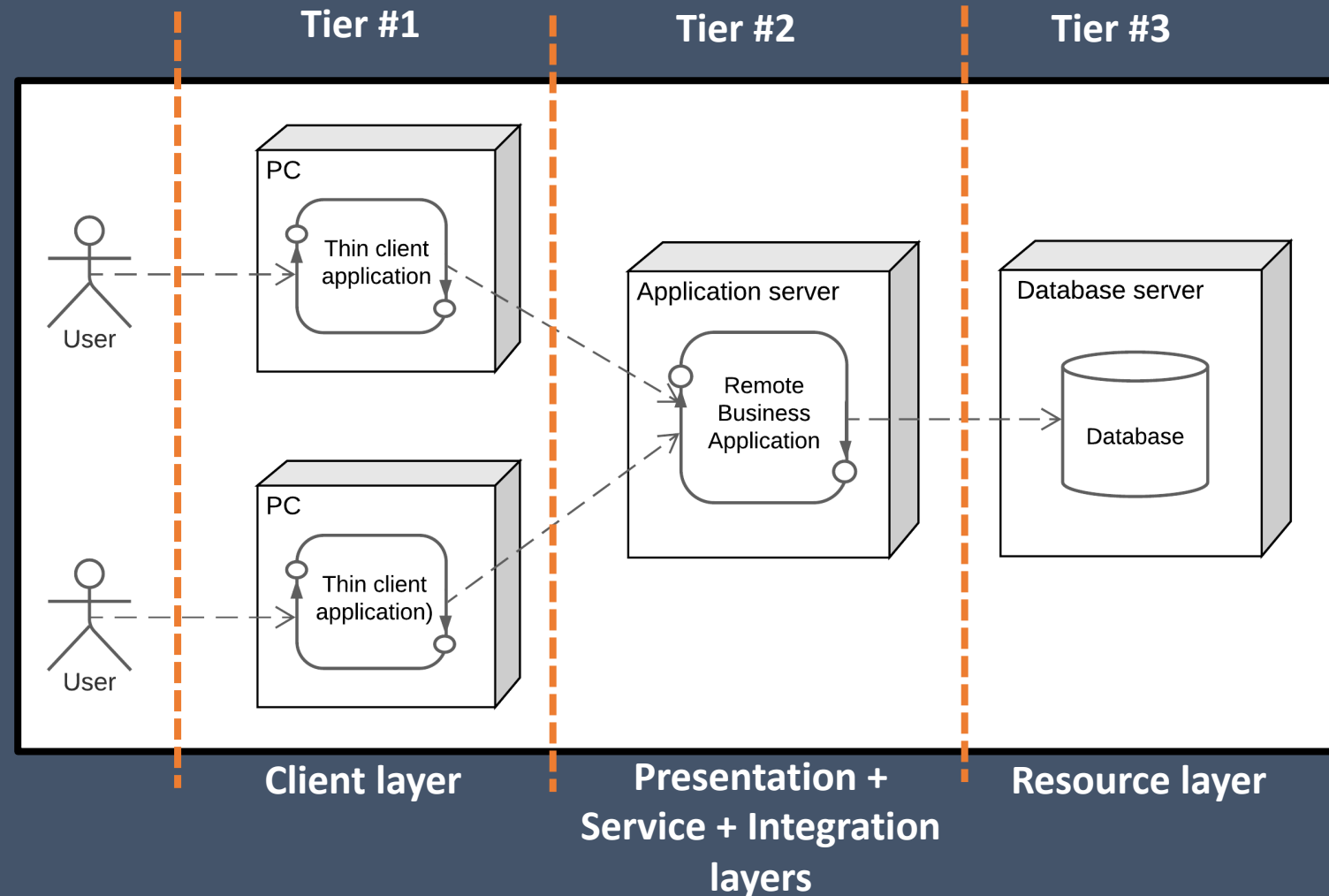
Typical layers in software development

- **Client, Presentation, Service (business), Integration (persistence), Resources.**
 - ✓ The concept of layers is technology agnostic.
- **Jakarta EE example:**
 - 2 tiers, 5 layers.



Typical tiered software architectures

- 1-tier
 - Standalone.
- 2-tiers
 - ✓ Thick client.
 - ✓ Thin client.
 - ✓ Thin client, web centric.
- N-tiers
 - Thin client.
 - ✓ Thin client, web centric.
 - ✓ Enterprise.



**We will discuss several types
of software architectures later in this course.**

Questions?